

PROJECT SYNOPSIS

CREATING IAM GROUPS AND ASSIGNING PERMISSIONS

Group-Based Access Control for AWS Accounts Using IAM Groups, Policies and Users

Project Domain : Cloud Computing / Amazon Web Services (AWS)

Project Type : Academic Project

Submitted by :

Sr. no.	Team Member	Enrollment ID
1	SHIVA SINGH	2415001488
2	ABHISHEK VERMA	2415000063
3	SHAILLY BHARDWAJ	12515990032
4	ADITYA SINGH	2415000115

Technology Stack :

AWS Identity and Access Management (IAM) • IAM User Groups • IAM Policies (AWS-Managed) • IAM Users • AWS Management Console • IAM Access Analyzer • IAM Policy Simulator

Project Synopsis

1. Abstract

As organizations move workloads onto AWS, the number of individuals who need console or API access grows quickly, and granting permissions to each person one at a time quickly becomes error-prone and difficult to audit. AWS Identity and Access Management (IAM) addresses this by allowing permissions to be defined once, as a policy, and attached to a group rather than to individual users, so that access can be managed at the role level instead of the person level. This project proposes and demonstrates a complete, console-based workflow for creating IAM user groups, attaching AWS-managed permission policies to those groups, creating individual IAM users, and assigning those users to the correct group so that they inherit the right level of access automatically.

In the proposed workflow, two IAM groups are created inside a single AWS account — Developers and Admins — and each is attached to a distinct set of AWS-managed policies: the Developers group receives AmazonEC2FullAccess and AmazonS3ReadOnlyAccess, while the Admins group receives AdministratorAccess. Two IAM users, User1 and User2, are then created with zero group memberships, after which User1 is mapped to the Developers group and User2 is mapped to the Admins group. The project verifies, directly in the console, that each user immediately inherits the exact permissions of the group they were added to, without any policy having been attached to the user individually.

The result is a demonstrable, reference-pattern access-control setup that shows how group-based permission management scales cleanly as an organization adds more users, keeps access auditable through consistent naming and group membership, and reduces the risk of permission drift that comes from managing policies on a per-user basis.

Keywords

- AWS
- AWS IAM
- IAM Groups
- IAM Policies
- IAM Users
- Least Privilege
- Role-Based Access Control
- Access Analyzer

2. Project Summary

The project is focused on designing a group-based access-control structure inside AWS IAM — two permission groups, Developers and Admins, each carrying a distinct set of AWS-managed policies — and then provisioning individual IAM users who are mapped into the correct group rather than being granted permissions directly. The final environment is intended to demonstrate a working, verifiable access model: a user's effective permissions are shown to come entirely from their group membership, and changing that membership is shown to be the single point of control for what the user can and cannot do in the account.

3. Introduction

AWS Identity and Access Management (IAM) is the service that controls who can authenticate into an AWS account and what actions they are authorized to perform once authenticated. In accounts with only a handful of users, permissions are sometimes attached directly to each IAM user, but this approach does not scale: as the number of users grows, the same policy has to be attached, reviewed and eventually revoked separately for every individual, and it becomes difficult to answer a simple question such as "who currently has EC2 access?" without checking every user one by one.

IAM user groups solve this by acting as a named container for a set of policies. A policy is attached once, to the group, and every user who is a member of that group automatically inherits it. Adding a new team member with the same responsibilities becomes a single action — adding them to the group — and removing access becomes equally simple. This project builds and verifies that exact pattern end to end, using two representative roles, Developers and Admins, to show how differentiated access levels are enforced purely through group membership.

3.1 Need for the Project

- To understand how IAM groups, policies and users relate to one another inside an AWS account.
- To learn the difference between attaching a policy to an individual user versus attaching it to a group.
- To configure AWS-managed policies correctly for two distinct roles with different access levels.
- To practically verify that a user's permissions come from their group membership, rather than assuming a configuration is correct.
- To understand least-privilege access design by giving each group only the policies its role needs.
- To gain hands-on experience with AWS IAM console workflows.

3.2 Scope

The scope covers a single AWS account: opening the IAM console, creating two user groups (Developers and Admins), attaching AWS-managed policies to each group, creating two IAM users (User1 and User2), assigning each user to the appropriate group, and validating that each user's effective permissions match the policies attached to their group. The project is designed as an academic demonstration and can later be extended with custom (inline) policies, permission boundaries, multiple nested teams, IAM roles for cross-account access, and Infrastructure-as-Code automation of the entire group and policy structure.

4. Problem Statement

An AWS account with multiple users, each needing a different level of access, cannot be managed safely by attaching policies to users individually: the same policy has to be reattached for every new user with the same role, revoking access on offboarding requires remembering every policy that was ever attached to that specific user, and there is no single place to see which users share a given permission level. Simply granting every user AdministratorAccess would solve the access problem but would violate least privilege and create an unnecessarily large blast radius if any single set of credentials were compromised.

The project addresses this problem by introducing IAM groups as an intermediary layer between policies and users: policies are attached to a group that represents a role — Developers or Admins — and users are simply added to the group that matches their responsibilities. Because a user's permissions come entirely from their group membership, changing a user's role is a single group-reassignment action rather than a policy audit, and this behaviour is validated directly rather than assumed.

5. Objective

- Understand the fundamentals of IAM groups, IAM policies and IAM users, and how they relate to one another.
- Access the IAM console and review the IAM dashboard.
- Create a "Developers" user group with zero attached policies initially.
- Attach AmazonEC2FullAccess and AmazonS3ReadOnlyAccess to the Developers group.
- Create an "Admins" user group and attach AdministratorAccess to it.
- Create two individual IAM users, User1 and User2, with no group memberships at creation.
- Assign User1 to the Developers group and verify inherited EC2 and S3 permissions.
- Assign User2 to the Admins group and verify inherited AdministratorAccess.
- Review best practices for least-privilege, naming and periodic access review.

5.1 Group and Policy Configuration

Parameter	Developers Group	Admins Group
Group Name	Developers	Admins
Attached Policy	AmazonEC2FullAccess, AmazonS3ReadOnlyAccess	AdministratorAccess
Access Level	Full EC2 control, read-only S3	Full administrative access to the account
Assigned User	User1	User2

6. Technologies and AWS Services Used

Component	Role in Project
AWS IAM	Provides the central identity and access-management layer for the account.
IAM User Groups	Act as named containers for policies (Developers, Admins) so permissions are managed once per role.
IAM Policies	AWS-managed JSON documents (AmazonEC2FullAccess, AmazonS3ReadOnlyAccess, AdministratorAccess) that define allowed actions.
IAM Users	Individual identities (User1, User2) representing people who need console or API access.
AWS Management Console	Used to perform and document every configuration step.
IAM Access Analyzer	Reviewed as a best practice for identifying unused or unintended access.
IAM Policy Simulator	Reviewed as a best practice for testing the effect of a policy before or after assignment.

6.1 Why Groups Rather Than Per-User Policies

Attaching AmazonEC2FullAccess and AmazonS3ReadOnlyAccess directly to User1, and AdministratorAccess directly to User2, would technically produce the same permissions but would not scale: a third developer joining the team would require the same two policies to be located and reattached by hand, and there would be no single object in the account that represents "the set of permissions a developer has." Attaching policies to the Developers and Admins groups instead means

the policy set exists in exactly one place, and any user added to that group inherits it immediately and consistently.

6.2 Why Separate Groups for Developers and Admins

Using two separate groups, rather than one shared group with every policy attached, keeps each role's access level explicit and independently auditable. The Developers group's policies (EC2 full access, S3 read-only) and the Admins group's policy (AdministratorAccess) cannot be confused with one another, and a reviewer can determine a user's full permission set simply by checking which group or groups that user belongs to, without inspecting policies attached to the user directly.

7. Proposed Solution / Architecture

The proposed access-control structure places two IAM groups inside a single AWS account. The Developers group carries AmazonEC2FullAccess and AmazonS3ReadOnlyAccess, representing a role that needs to manage compute resources and read data but not administer the account. The Admins group carries AdministratorAccess, representing a role that needs full control. Two IAM users are created independently of any group and are only granted access once explicitly assigned to the group matching their role.

7.1 Logical Access Flow

A user's effective permissions flow as: IAM User → Group Membership → Policies Attached to Group → Effective Permissions. User1 is added to Developers and therefore inherits AmazonEC2FullAccess and AmazonS3ReadOnlyAccess; User2 is added to Admins and therefore inherits AdministratorAccess. No policy is ever attached to User1 or User2 directly — every permission each user holds can be traced back to a single group membership.

7.2 Implementation Phases

Phase	Focus	Steps
01 – Console Access	Open the IAM dashboard	Step 1
02 – Group Creation	Create the Developers and Admins user groups	Steps 2, 4
03 – Policy Attachment	Attach AWS-managed policies to each group	Steps 3, 4
04 – User Provisioning	Create individual IAM users	Step 5
05 – Assignment & Verification	Map users to groups and confirm inherited access	Steps 6, 7

7.3 Core Design Principle

The architecture separates two responsibilities that are easy to conflate: identity (who a user is) and permission (what a role is allowed to do). IAM users represent identity, while IAM groups and their attached policies represent permission. Because permission lives on the group rather than the user, a user's access level can be changed completely by changing their group membership alone, without editing a single policy document.

8. Project Workflow and Implementation

Step 1 – Access the IAM Console: Signed in to the AWS Management Console and searched for “IAM” in the top navigation bar. Selecting the IAM service opened the IAM Dashboard, which summarizes account security status and resource counts.

Step 2 – Create the “Developers” User Group: From IAM → User groups, the account started with zero groups. Create group was selected, the group was named Developers, and AWS confirmed creation immediately.

Step 3 – Attach Permissions to Developers: AmazonEC2FullAccess (full control over EC2 instances and resources) and AmazonS3ReadOnlyAccess (view and read objects across S3 buckets) were attached to the Developers group. Any user later added to this group automatically inherits both policies, with no per-user setup needed.

Step 4 – Create the “Admins” Group: A second group, Admins, was created the same way. The IAM user groups list then showed both groups, and AdministratorAccess was attached to Admins for full account privileges.

Step 5 – Create IAM Users: With groups and policies ready, individual IAM users were created next — User1 and User2 — each starting with zero group memberships until explicitly assigned.

Step 6 – Assign User1 to the Developers Group: On User1’s Groups tab, Add user to groups was selected, Developers was checked, and the assignment was confirmed. User1 immediately inherited the group’s EC2 and S3 permissions.

Step 7 – Assign User2 to the Admins Group: The same flow was applied to User2: Admins was checked on the Add user to groups screen. User2 then held AdministratorAccess through group membership alone.

8.1 Expected Sequence

IAM Console Opened → Developers Group Created → EC2 & S3 Policies Attached to Developers → Admins Group Created → AdministratorAccess Attached to Admins → User1 and User2 Created → User1 Assigned to Developers → User2 Assigned to Admins → Inherited Permissions Verified for Both Users.

9. Detailed Component Roles

9.1 IAM User Groups

Developers and Admins act as the single point of control for permissions in this project; every policy used in the project is attached to one of these two groups rather than to a user.

9.2 IAM Policies

AmazonEC2FullAccess, AmazonS3ReadOnlyAccess and AdministratorAccess are the AWS-managed policy documents that define exactly which actions are allowed for each group, without requiring either group's policy to be written from scratch.

9.3 IAM Users

User1 and User2 represent individual identities in the account; each is created with no permissions of its own and receives access purely through the group it is later assigned to.

9.4 AWS Management Console

Used throughout the project to create groups, attach policies, create users and assign group membership, and to document each step with a corresponding screenshot.

10. Access Control and Security

All access decisions in this project are enforced at the group level, with User1 and User2 never receiving a policy directly. The following practices were applied and are recommended for production use:

- Attach policies to groups, not to individual users, so access can be reviewed and revoked in one place.
- Give each group only the policies its role actually needs — Developers does not receive AdministratorAccess.
- Use clear, role-based group names (Developers, Admins) so audits do not require inspecting individual policies.
- Review access periodically with IAM Access Analyzer to identify unused permissions.
- Use the IAM Policy Simulator before attaching a new policy to a group, to confirm its effect in advance.
- Log IAM changes using AWS CloudTrail so group membership and policy changes are auditable.

11. Testing and Validation

Testing was built into the final phase of the workflow itself: each access-control claim (a user has exactly the permissions of their group, and no more) was validated by checking the user's Groups and Permissions tabs directly, rather than being inferred from configuration alone.

ID	Test Action	Expected Result
TC-01	Open the IAM Dashboard	Dashboard loads and shows account security summary
TC-02	Create Developers group	Group appears in the User groups list
TC-03	Attach EC2 and S3 policies to Developers	Both policies show under the group's Permissions tab
TC-04	Create Admins group	User groups list shows 2 groups
TC-05	Attach AdministratorAccess to Admins	Policy shows under the Admins Permissions tab
TC-06	Create User1 and User2	Both users show 0 group memberships initially
TC-07	Assign User1 to Developers	User1's Groups tab shows Developers
TC-08	Assign User2 to Admins	User2's Groups tab shows Admins
TC-09	Verify User1's effective permissions	User1 shows EC2 full access and S3 read-only access
TC-10	Verify User2's effective permissions	User2 shows AdministratorAccess

11.1 Monitoring Checklist

- Number of user groups present at each workflow stage
- Policies attached to each group (Developers, Admins)
- Group-membership count on each IAM user before and after assignment
- Effective permissions shown on each user's Permissions tab
- IAM Access Analyzer findings for unused or unintended access

11.2 Success Criteria

The project is considered successful when User1, having been added only to the Developers group, is shown to hold exactly AmazonEC2FullAccess and AmazonS3ReadOnlyAccess, and User2, having been added only to the Admins group, is shown to hold exactly AdministratorAccess — with neither user having any policy attached directly.

12. Challenges and Troubleshooting

Problem	Troubleshooting Approach
Newly created group shows no policies	Confirm the policy was actually attached and saved on the group's Permissions tab, not just searched for.
User does not appear to have inherited permissions	Re-check the user's Groups tab to confirm group membership was saved, and refresh the Permissions tab.
Wrong policy attached to a group	Detach the incorrect AWS-managed policy from the group and attach the correct one; changes apply to all members immediately.
User has more access than expected	Check whether the user belongs to more than one group or has a policy attached directly to the user.
AdministratorAccess accidentally attached to Developers	Detach AdministratorAccess from Developers immediately and re-verify against the intended policy list.

13. Advantages

- Grants permissions to many users through a single group, instead of repeating policy attachment per user.
- Makes a user's access level fully traceable to their group membership.
- Simplifies onboarding and offboarding — adding or removing a user from a group instantly changes their access.
- Uses AWS-managed policies, requiring no custom policy JSON to be written or maintained.
- Supports least-privilege design by scoping each group to only the access its role needs.
- Keeps audits simple through clear, role-based group naming.

14. Limitations

- A user can belong to multiple groups, so effective permissions must be checked across all of a user's group memberships, not just one.
- AWS-managed policies (like AmazonEC2FullAccess) grant broader access than a tailored custom policy might; production use may need scoped custom policies instead.
- The project is scoped to two groups and two users, not a full organizational role hierarchy.
- IAM configuration alone does not address credential hygiene, such as MFA enforcement or access-key rotation.
- AWS resources referenced in this project (EC2, S3) may incur charges if actually provisioned and used beyond the free tier.

15. Expected Results and Outcomes

Condition	Expected Observation
Developers group created	Group appears in User groups list with 0 members initially
Policies attached to Developers	AmazonEC2FullAccess and AmazonS3ReadOnlyAccess listed under Permissions
Admins group created	User groups list shows 2 groups total
AdministratorAccess attached to Admins	Policy listed under Admins' Permissions tab
User1 and User2 created	Both users show 0 groups on creation
User1 assigned to Developers	User1's effective permissions match the Developers group exactly
User2 assigned to Admins	User2's effective permissions match AdministratorAccess exactly

16. Future Scope

- Replace AWS-managed policies with tailored custom policies scoped to specific resources.
- Introduce IAM permission boundaries to cap the maximum permissions a group can ever grant.
- Automate group, policy and user creation using AWS CLI or Infrastructure-as-Code (CloudFormation/Terraform).
- Add additional role-based groups (e.g., Auditors, Support) as the organization grows.
- Enforce MFA and periodic access-key rotation for all IAM users.
- Integrate IAM Access Analyzer findings into a recurring access-review process.

17. Project Deliverables

- Two IAM user groups (Developers, Admins) with AWS-managed policies attached.
- Two IAM users (User1, User2) created and mapped to the correct group.
- Verified evidence that each user's effective permissions match their group's attached policies.
- Console screenshots documenting all seven workflow steps.
- Final project synopsis, assignment report and presentation.
- Version-controlled repository containing all project artifacts.

18. Conclusion

This project demonstrates a group-based access-control pattern in AWS IAM, showing how permissions attached to a small number of role-based groups can be inherited consistently by any number of users. By creating the Developers and Admins groups, attaching AWS-managed policies to each, provisioning IAM users independently of any group, and validating that each user's effective permissions come only from their group membership, the project demonstrates identity management, least-privilege policy design, and auditable access control. The architecture provides a foundation for custom policies, permission boundaries, and Infrastructure-as-Code automation of IAM in a production account.

19. References / Learning Resources

- Amazon Web Services (AWS) documentation for IAM.

- Amazon Web Services documentation for IAM user groups.
- Amazon Web Services documentation for AWS-managed and customer-managed policies.
- Amazon Web Services documentation for IAM Access Analyzer.
- Amazon Web Services documentation for the IAM Policy Simulator.
- Amazon Web Services best-practices guide for identity and access management.

End of Project Synopsis